

2. What is the Common Language Runtime (CLR)?

Brief summary: The Common Language Runtime is a runtime environment that manages the execution of the .NET applications.

The **Common Language Runtime** is a runtime environment that manages the execution of .NET applications. The CLR works as a special "operating system" for .NET applications, that manages all operations (like memory management) that otherwise must have been dealt with by a programmer. The CLR stands between the actual operating system (for example Windows) and the application.

Before we move on, I would like to say one thing: you may think that CLR is not a junior-level topic, and you might be right - this is a pretty low-level feature of .NET and you certainly can start programming .NET applications without even knowing that the CLR exists. Nevertheless, I wanted to introduce this topic as throughout this course I mention the CLR very often, as it affects many aspects of .NET programming. I want to make sure every subject in this course is explained in detail, even if it sometimes exceeds the junior level. **Some of the topics of this course simply can't be understood thoroughly without a basic understanding of the role of the CLR.**

All right, let's move on then. The important thing to understand is that the CLR is the .NET component that is not exclusive for C# applications. All programs written for the .NET are executed by the CLR. All code executed under the CLR is called the **managed code**. Thanks to the CLR, cross-language integration is supported in .NET. For example, you can have a C#'s class derived from a class defined in F#, because the CLR can understand both languages (because both are compiled to the Intermediate Language).

The CLR is responsible for many operations essential for any .NET application to work. Some of them are:

- **JIT (Just-in-time) compilation** - the compilation of the Common Intermediate Language to the binary code. Thanks to that the .NET applications can be used cross-platform because the code is compiled to platform-specific binary code only right before execution. See the "What is the Common Intermediate Language (CIL)?" lecture for more information on that.

- **Memory management** - CLR allocates the memory needed for every object created within the application. CLR also includes the Garbage Collector, which is responsible for releasing and defragmenting the memory. See the "What is the Garbage Collector?" lecture for more information.
- **Exception handling** - when the exception is thrown, the CLR makes sure the code execution is redirected to the proper catch clause. See the "How are exceptions handled in C#?" lecture for more information.
- **Thread management** - threads are beyond junior level, so let's just shortly say that the CLR manages the execution of the multi-threaded applications, making sure all threads work together well
- **Type safety** - part of the CLR is the **CTS - Common Type System**. CTS defines the standard for all .NET-compatible languages. Thanks to that, the CLR can understand types defined in C#, F#, Visual Basic, and so on, enabling cross-language integration.
- And many more

The CLR is the implementation of the CLI - **Common Language Infrastructure**. CLI was originally created by Microsoft and is standardized by ISO and ECMA. Sounds confusing? Let's explain it in this way - CLI is like a design of a house - it describes where walls, windows, piping, and electricity goes. It was originally created by Microsoft. ISO and ECMA are like state authorities who approve the design and make sure it is safe and reasonable, and that houses built based on this design will all function properly. Using this design, Microsoft builds a house. In this metaphor, the design is the CLI and the house built by Microsoft is the CLR. The thing about designs is that we can build many things based on the same design. That means another company could implement its own version of the project in accordance with the CLI. Does it ever happen? Actually, it does! For example, there is Mono Runtime, a counterpart of the CLR developed by a company called Ximian.

By now you should have a general idea of what the CLR does, but before we wrap up let's go **step-by-step** through a (simplified) process of creating and running the application to see when and how the CLR plays its role.

1. The programmer writes the program, which at first is just a bunch of text files.
2. The compiler compiles the text file to Common Intermediate Language, which is platform-independent. The compiler also prepares the metadata that describes all the types along with the methods they include.
3. Now the CLR comes into play. It starts the program under the specific operating system.
4. The CLR's Just-In-Time compiler compiles the Intermediate Language to binary code that can be interpreted by the machine's operating system. It uses the metadata prepared by the compiler.

5. As the application runs, the CLR manages all its low-level aspects - memory management, threads, exceptions handling, and so on.
6. When the application stops, the CLR's job is done.

As you can see, the **CLR is the critical component of .NET**. It manages basically everything that happens under the hood of the running application, allowing us, the programmers, to focus on business aspects of the development. Before tools like the CLR were introduced, programmers must have dealt with all things like memory management, which is the case in languages like C (can you imagine allocating memory for the objects by hand?).

Tip: other interview questions on this topic:

- **"What is the difference between CLR, CLI, and CIL?"**
CLR (Common Language Runtime) is an implementation of the CLI (Common Language Infrastructure). CIL is Common Intermediate Language, to which all .NET-compatible languages get compiled.
- **"What is the CTS?"**
CTS is the Common Type System, which is a standardized type system for all .NET-compatible languages, which makes them interoperable - for example, we can have a C# class derived from an F# class.
- **"Is the CLR the only implementation of the CIL?"**
No. Anyone can create their implementation of the CIL. One of the examples is Mono Runtime.